

Explainable artificial intelligence (XAI)

Olivier Caelen

Introduction

For many people: *most of machine learning models* == “*black boxes models*”

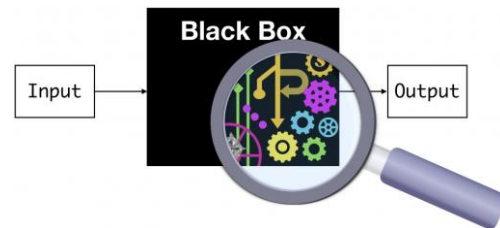
→ They can make good predictions, but you can't easily understand the logic behind those predictions.

The interpretation of complex "black box" Machine Learning (ML) predictive models is an important issue in the field of Data Science today.

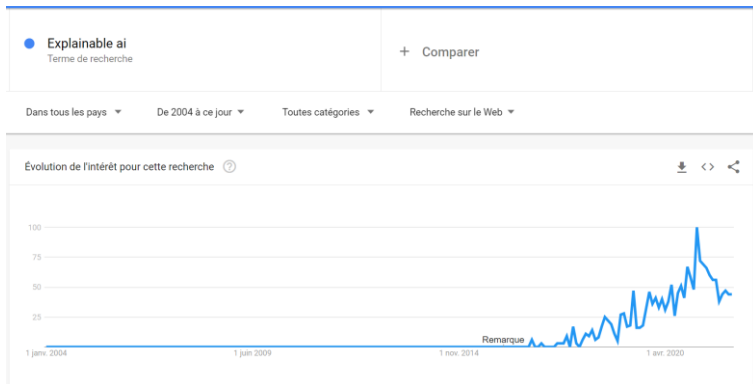
We need to be able to explain the predictions, for instance in the following cases :

- **Legal aspect:** Article 22 of the GDPR provides rules to prevent humans from being subjected to decisions made only by machines. Models without an explanation may incur a penalty of up to 20,000,000 € or, in the case of a company, 4% of the total worldwide turnover of the previous year (the higher amount being taken).
- **Model validation:**
 - The model has a good precision, but we seek to know the influencing variables in order to check the coherence with the business knowledge of the domain. It help to debug our model.
 - For some applications, we need to control the risk of the model, which requires a deep understanding of it.
- **Explanation and recommendation:** Imagine a client of a bank requesting a loan, the request is refused following the decision of an ML model. However, it is necessary to be able to explain to this client the reasons for this refusal in order to be able to some advises/recommendations to him.

The interpretability of a predictive model is like a label on a medicine bottle. We need to make our drug pills effective and transparent for easy adoption.



Why XAI?



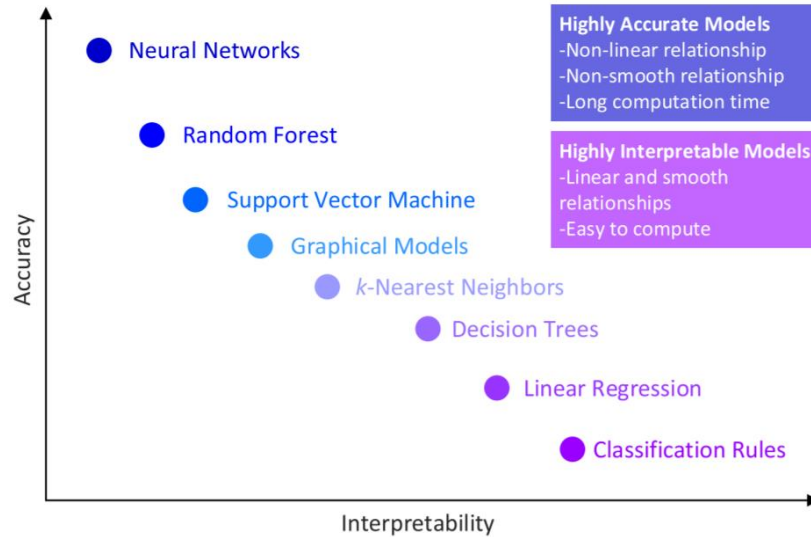
- The field of explainable artificial intelligence (XAI) is not new. Its beginnings go back to the origins of AI research.
- Since around 2015, there has been a resurgence in the areas of XAI research. It parallels the progress and increasing presence of ML systems in society.
- As the adoption of ML systems continues to increase and reach new audiences, increasingly complex models such as DL neural models are emerging. This brings new challenges concerning explainability to engineers and those involved in the models.

Impact of XAI for different types of stakeholders

Engineers	Consumers	Regulators
Increase Understanding	Increase Trust	Increase Trust
Improve Performance	Bias & Transparency	Bias & Transparency
Create Better Algorithms	Understand Impact	Compliance
Product Complex Models	Report & Analyses	Report

Understanding the behavior of a model is essential for many tasks:

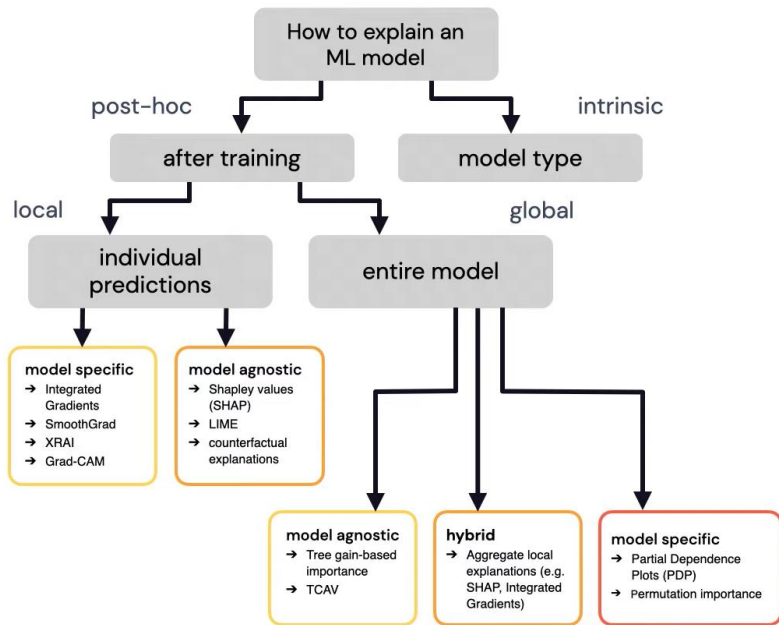
- Explain predictions to support decision making processes.
- Debug unexpected behaviors of a model.
- Refine modeling and data collection processes.
- Verify that a model behavior is acceptable .
- Present model predictions to stakeholders.



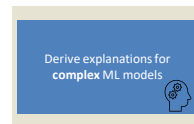
Credit: Machine Learning for 5G/5G Mobile and Wireless Communications: Potential, Limitations, and Future Directions.
MOROCHO-CAYAMCELA et al.

The trade-off between interpretability and accuracy of some relevant ML models. Highly interpretable algorithms such as classification rules, or linear regression, are often inaccurate. Very accurate DNNs are a classic example of black boxes

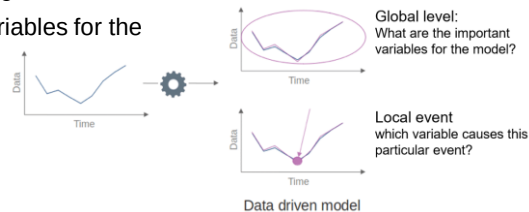
Taxonomy: explainable ML models



- After training (Post-hoc)
 - Applying methods that analyze models after training
- Model type (Intrinsic)
 - Restricting the complexity of the machine learning model
 - Simple Structure: decision trees, linear models



- Individual predictions (Local)
 - Interpretability of individual predictions or a small part of the model's prediction space
- Entire model (Global) – a.k.a. “feature importance”
 - Aggregation/ranked contributions of input variables for the entire model's prediction space



- Model Specific
 - Only works for specific models due to definition
- Model Agnostic
 - Portable across model definitions
- Hybrid
 - Aggregate the scores of some local methods to provide a global view of the model's prediction space

Model for the experiments

```
import pandas as pd

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn import ensemble

✓ 0.4s

housing = fetch_california_housing()

X = pd.DataFrame(data=housing['data'], columns=housing['feature_names'])
y = housing['target']

X_tr, X_vl, y_tr, y_vl = train_test_split(X, y, test_size=0.2, random_state=42)
✓ 0.5s

X_tr
✓ 0.8s
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
14196	3.2596	33.0	5.017657	1.006421	2300.0	3.691814	32.71	-117.03
8267	3.8125	49.0	4.473545	1.041005	1314.0	1.738095	33.77	-118.16
17445	4.1563	4.0	5.645833	0.985119	915.0	2.723214	34.66	-120.48
14265	1.9425	36.0	4.002817	1.033803	1418.0	3.994366	32.69	-117.11
2271	3.5542	43.0	6.268421	1.134211	874.0	2.300000	36.78	-119.80
...	...	...	...	...	...	...	...	...
11284	6.3700	35.0	6.129032	0.926267	658.0	3.032258	33.78	-117.96
11964	3.0500	33.0	6.868597	1.269488	1753.0	3.904232	34.02	-117.43
5390	2.9344	36.0	3.986717	1.079696	1756.0	3.332068	34.03	-118.38
860	5.7192	15.0	6.395349	1.067979	1777.0	3.178891	37.58	-121.96
15795	2.5755	52.0	3.402576	1.058776	2619.0	2.108696	37.77	-122.42

16512 rows x 8 columns

```
y_tr
✓ 0.5s
array([1.03 , 3.821, 1.726, ..., 2.221, 2.835, 3.25 ])
```

```
params = {
    "n_estimators": 500,
    "max_depth": 4,
    "min_samples_split": 5,
    "learning_rate": 0.01,
    "loss": "squared_error",
}

reg = ensemble.GradientBoostingRegressor(**params, random_state=42)
reg.fit(X_tr, y_tr)

mse_tr = mean_squared_error(y_tr, reg.predict(X_tr))
print("The mean squared error (MSE) on training set: {:.4f}".format(mse_tr))

mse_vl = mean_squared_error(y_vl, reg.predict(X_vl))
print("The mean squared error (MSE) on validation set: {:.4f}".format(mse_vl))

✓ 8.5s

The mean squared error (MSE) on training set: 0.2585
The mean squared error (MSE) on validation set: 0.2954
```

Attribute Information

- MedInc: median income in block group
- HouseAge: median house age in block group
- AveRooms: average number of rooms per household
- AveBedrms: average number of bedrooms per household
- Population: block group population
- AveOccup: average number of household members
- Latitude: block group latitude
- Longitude: block group longitude

The target variable is the median house value for California districts, expressed in hundreds of thousands of dollars (\$100,000)

Permutation Importance

What input variables does your model consider important?

Permutation Importance

- Feature importance are global technics (i.e., entire model) that assess input features based on their usefulness in predicting a target variable for the entire model's prediction space.
- There are several ways to measure the importance of an input feature. In this section, we'll focus on *permutation importance*.
- Permutation importance is a model inspection technique that can be used for any fitted estimator when the data is tabular. This is especially useful for non-linear or opaque predictive model.
- Compared to most other approaches, *permutation importance* is
 - fast to calculate
 - widely used and understood
 - model agnostic
- The permutation importance is defined to be the decrease in a model score when a single feature value is randomly shuffled. The drop in the model score is indicative of how much the model depends on the feature.

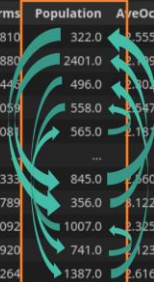
Warning: Features that are deemed of **low importance for a bad model** (low cross-validation score) could be **very important for a good model**. Therefore it is always important to evaluate the predictive power of a model using a held-out set (or better with cross-validation) prior to computing importances. Permutation importance does not reflect to the intrinsic predictive value of a feature by itself but **how important this feature is for a particular model**.

Credit: https://scikit-learn.org/stable/modules/permutation_importance.html

How It Works

Permutation importance is calculated after a predictive model has been fitted (i.e., we will not change the model).

If I randomly shuffle a single column of a validation dataset (e.g., *Population*), leaving the target and all other columns in place, how would that affect the accuracy of the predictions in this now mixed data?



	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	5.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	5.19842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073445	496.0	5.301260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	5.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	5.187467	37.85	-122.25
...	...	...	...	...	...	...	...	...
20635	1.5603	25.0	5.045455	1.133333	845.0	5.60606	39.48	-121.09
20636	2.5568	18.0	6.114035	1.315789	356.0	5.122807	39.49	-121.21
20637	1.7000	17.0	5.205543	1.120092	1007.0	5.325635	39.43	-121.22
20638	1.8672	18.0	5.329513	1.171920	741.0	5.123209	39.43	-121.32
20639	2.3886	16.0	5.254717	1.162264	1387.0	5.616981	39.37	-121.24

Randomly rearranging a single column should result in less accurate predictions, as the resulting data no longer matches anything observed in the real world.

The accuracy of the model suffers especially if we mix a column that the model relied heavily on for predictions.

the process is as follows:

1. Get a trained model.
2. Mix values in one column, make predictions using the resulting data set. Use these predictions and the true target values to calculate how much the loss function has suffered from the reshuffle. This deterioration in performance is a measure of the importance of the variable you just mixed up.
3. Return the data to the original order (cancel random play in step 2). Now repeat step 2 with the next column in the dataset, until you have calculated the importance of each column.

Code Example

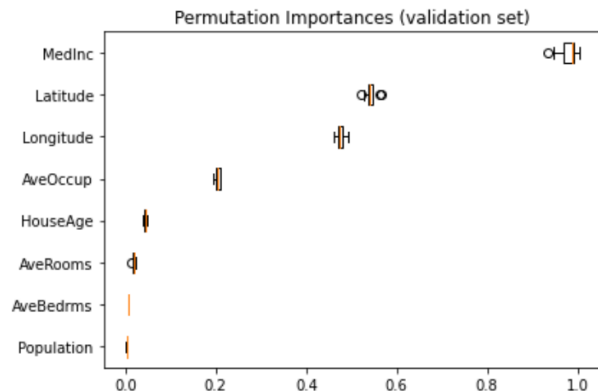
```
from sklearn.inspection import permutation_importance
import matplotlib.pyplot as plt

res_pi = permutation_importance(reg, X_vl, y_vl, n_repeats=10, n_jobs=2,
                               scoring='neg_mean_squared_error', random_state=42)

sorted_idx = res_pi.importances_mean.argsort()

fig, ax = plt.subplots()
ax.boxplot(
    res_pi.importances[sorted_idx].T, vert=False, labels=X_vl.columns[sorted_idx]
)
ax.set_title("Permutation Importances (validation set)")
fig.tight_layout()
plt.show()
```

✓ 8.8s



Interpreting Permutation Importance

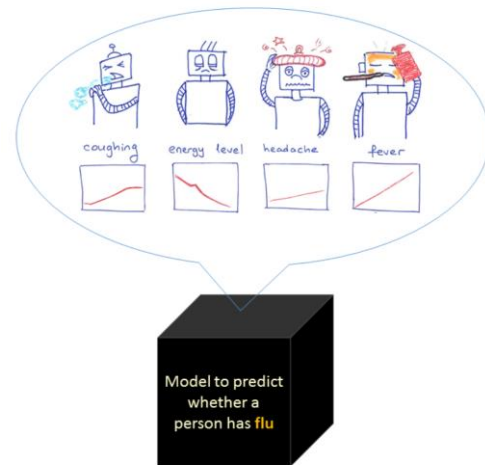
- Variables at top are the most important features, and those towards the bottom matter least.
- The values in the X-axis shows how much model performance decreased with a random shuffling (in this case, using "neg_mean_squared_error" as the performance metric)
- There is an amount of randomness in a column mixing. We measure this degree of randomness in our calculation of the Permutation Importance by repeating the process with several shuffles (here 10). Box plots show how performance varied from one-reshuffling to the next.
- Sometimes you will see negative values for the permutation importance. In these cases, the predictions on the mixed data were found to be more accurate than the actual data. This happens when the variable didn't matter (should have been near 0 importance), but chance made predictions on mixed data more accurate.

Partial dependence plots (PDP)

How does each feature affect your predictions?

Partial Plots

- While feature importance shows which variables affect predictions the most, partial dependence plot shows how a feature affects predictions.
- This is useful for answering questions such as: taking all the other housing characteristics under control, what impact do *median income* (MedInc) on house prices?
- Partial dependence plots can show whether the relationship between the target and a feature is linear, monotonic or more complex. E.g., when applied to a linear regression model, PDP always show a linear relationship.
- Due to the limitations of human perception, the size of the set of input features of interest should be small (usually, one or two).



How It Works

- Partial dependence works by marginalizing the model output over the distribution of the features in the complement set X_C .
- Let $X = (X_1, X_2, \dots, X_n)$ be a random vector of size n representing the input variables of the model h .
- Let X_S be the set of input variables of interest and let X_C be its complement. Usually, there are only one or two features in the set X_S .
- The partial dependence function for regression is defined as

$$\hat{f}_S(x_s) = E_{X_C}[h([x_s, x_C])] = \int h([x_s, x_C]) f_{X_C}(x_C) dx_C$$

where x_s are the features of interest and X_C are the other variables.

- Computing this integral for various values of x_s produces a PDP plot.
- Estimation for a dataset with N observations

$$\hat{f}_S(x_s) \approx \frac{1}{N} \sum_{i=1}^N h([x_s, x_C^{(i)}])$$

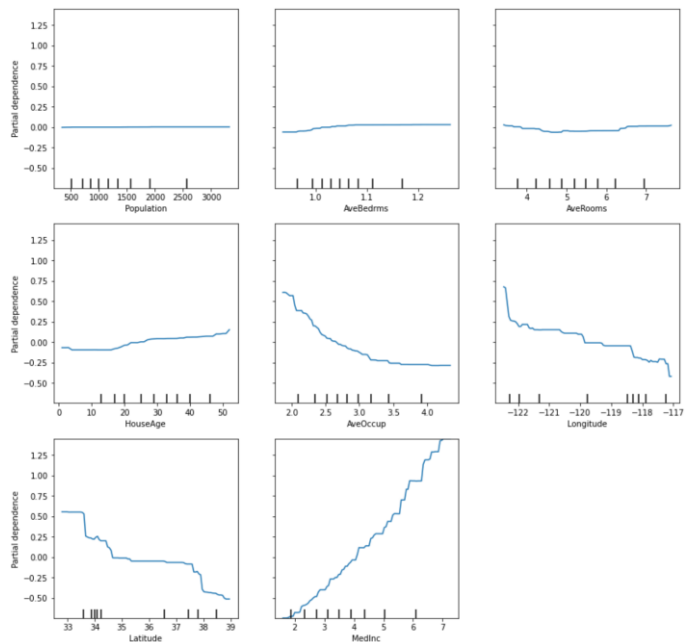
where $x_C^{(i)}$ are actual feature values from a dataset for the features in which we are not interested.

Code Example

```
from sklearn.inspection import PartialDependenceDisplay
import matplotlib.pyplot as plt

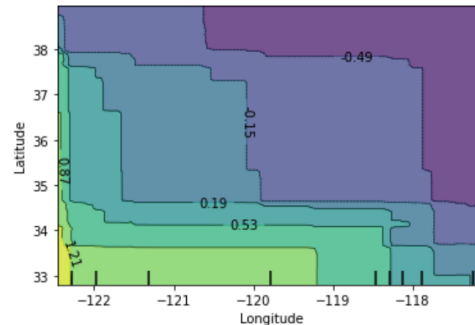
fig, ax = plt.subplots(figsize=(14, 14))
PartialDependenceDisplay.from_estimator(reg, X_vl, sorted_idx, ax=ax)
plt.show()
```

✓ 1.6s



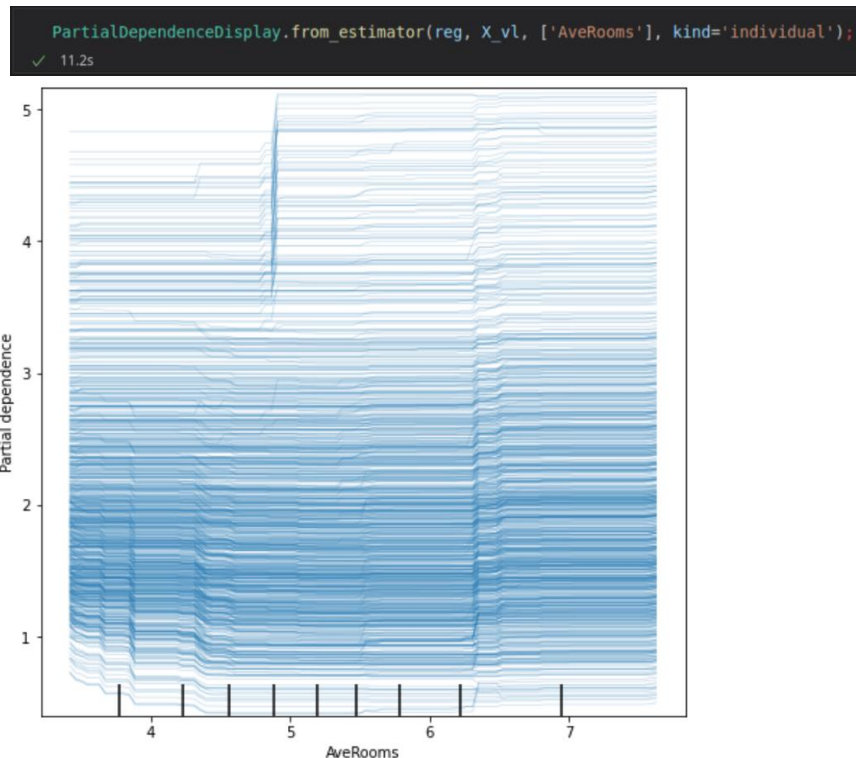
```
PartialDependenceDisplay.from_estimator(reg, X_vl, [['Longitude','Latitude']])
```

✓ 1.2s



Individual conditional expectation (ICE) plot

- Unlike a PDP, which shows the average effect of the input feature, an ICE plot visualizes the dependence of the prediction on a feature for each sample separately with one line per sample (i.e., each $h\left([x_s, x_C^{(i)}]\right)$ for all i).
- While PDPs are good at showing the average effect of target characteristics, they can mask a heterogeneous relationship created by interactions for other variables. When interactions are present the ICE plot will provide many more insights.

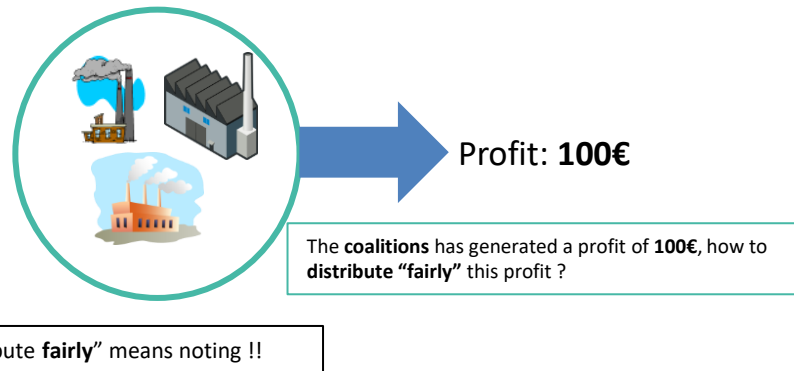


SHapley Additive exPlanations (SHAP) Value

Understand individual predictions

Cooperative game theory & Shapley value

- In a **cooperative** game, “**players**” have the possibility to **forge coalitions** to reach a **common goal**.
- A difficulty in cooperative game theory is **the distribution of profits among the players**.
- **Shapley value** provides a way to distribute **fairly** profits.
- We will identify some **axioms** that express properly what is “**fairly**”:
 - **Efficiency**
 - **Symmetry**
 - **Additivity**
 - **Dummy player**



In a cooperative game, the Shapley value is **the unique solution** having these following desirable properties.

$$\phi_i(N, v) = \frac{1}{|N|!} \sum_{S \subseteq N / \{i\}} |S|! (|N| - |S| - 1)! [v(S \cup \{i\}) - v(S)]$$

Shapley value – Notation & axioms

N is the set containing all the players

$$N = \{ \text{🏭}, \text{🏭}, \text{🏭} \}$$

S is a subset of N (i.e. $S \subseteq N$).

i is an element of N (i.e. $i \in N$)

v is a function that maps subsets of players to a real number (i.e. $v: 2^N \rightarrow \mathbb{R}$)

$v(S) \equiv$ "the revenue (cost) of the coalition S "

When a player i join a set of players S , the marginal contribution of player i to S is:

$$v(S \cup \{i\}) - v(S)$$

$\phi_i(N, v)$ is the Shapley value of the player i given the set N and the function v .

Efficiency – all the revenue (cost) of the grand coalition is redistributed among the players (no less, no more).

$$v(N) = \sum_{i \in N} \phi_i(N, v)$$

Symmetry – Players who bring the same contribution receive the same share.

$$\forall S \subseteq N \setminus \{i, j\}: v(S \cup \{i\}) = v(S \cup \{j\}) \Rightarrow \phi_i(N, v) = \phi_j(N, v)$$

Additivity – If you bring more, you get more.

$$\text{Let } v = v_1 + v_2 \text{ then } \phi_i(N, v) = \phi_i(N, v_1) + \phi_i(N, v_2)$$

Dummy player – Those who bring nothing receive nothing.

$$\forall S \subseteq N \setminus \{i\}: v(S \cup \{i\}) = v(S) \Rightarrow \phi_i(N, v) = 0$$

Theorem

Given a coalitional game (N, v) , there is a unique payoff division $x(v) = \phi(N, v)$ that divides the full payoff of the grand coalition and that satisfies the Symmetry, Dummy player and Additivity axioms: the Shapley Value

Lloyd Shapley

From Wikipedia, the free encyclopedia

For United States navy captain, see [Lloyd Stowell Shapley](#).

Lloyd Stowell Shapley (/ˈæpliː/; June 2, 1923 – March 12, 2016) was an American [mathematician](#) and [Nobel Prize-winning economist](#). He contributed to the fields of [mathematical economics](#) and especially [game theory](#).^{[1][2]}

Since the work of [von Neumann](#) and [Morgenstern](#) in 1940s, Shapley has been regarded by many experts as the very personification of game theory.^[3] With [Alvin E. Roth](#), Shapley won the 2012 [Nobel Memorial Prize in Economic Sciences](#) "for the theory of stable allocations and the practice of [market design](#)."

Contents [\[hide\]](#)

- 1 Life and career
- 2 Contribution
- 3 Awards and honors
- 4 Selected publications
- 5 See also
- 6 References

Lloyd Shapley



Shapley in 1980

Shapley value

$|S|!$ $\equiv$ number of permutation in S .

$(|N| - |S| - 1)!$ $\equiv$ number of permutation in $N \setminus (S \cup \{i\})$

An intuitively simple idea is: “compute the marginal contribution of a player to each of the possible coalitions, and then take their weighted average”.

$$\phi_i(N, v) = \frac{1}{|N|!} \sum_{S \subseteq N \setminus \{i\}} \underbrace{|S|!}_{\text{Weight}} \underbrace{(|N| - |S| - 1)!}_{\text{Weight}} \underbrace{[v(S \cup \{i\}) - v(S)]}_{\text{Marginal contributions}}$$

Another equivalent formulation

- Let N be a set with players: $N = \{P_1, P_2, P_3, P_4\}$
 - Let σ be the set with all the $4! = 24$ permutations of the indexes:
 - $\sigma = \{(1,2,3,4), (1,2,4,3), (1,4,2,3), (4,1,2,3), (4,1,3,2), (1,4,3,2), \dots, (4,3,2,1)\}$
- 24 permutations

The Shapley Value for the player i is defined as the average of its marginal contribution over all permutations:

$$\phi_i(N, v) = \frac{1}{|N|!} \sum_{\sigma} [v(S \cup \{i\}) - v(S)]$$

Where the σ runs over all the $|N|!$ permutations of the elements of N and S is the coalition of the predecessors of i .

Example 1

$\mathcal{S}$ is the coalition of the **predecessors** of i .

$$\phi_i(N, v) = \frac{1}{|N|!} \sum_{\sigma} [v(\mathcal{S} \cup \{i\}) - v(\mathcal{S})]$$


- $v(\{P_1\}) = 80$ $v(\{P_2\}) = 56$ $v(\{P_3\}) = 70$
- $v(\{P_1, P_2\}) = 80$ $v(\{P_1, P_3\}) = 85$ $v(\{P_2, P_3\}) = 72$
- $v(\{P_1, P_2, P_3\}) = 90$

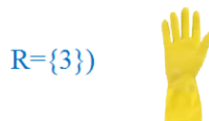
σ	$[v(\mathcal{S} \cup \{P_1\}) - v(\mathcal{S})]$	$[v(\mathcal{S} \cup \{P_2\}) - v(\mathcal{S})]$	$[v(\mathcal{S} \cup \{P_3\}) - v(\mathcal{S})]$
P_1, P_2, P_3	80 $\mathcal{S} = \phi$	0	10
P_1, P_3, P_2	80 $\mathcal{S} = \phi$	5	5
P_2, P_1, P_3	24 $\mathcal{S} = \{P_2\}$	56	10
P_2, P_3, P_1	18 $\mathcal{S} = \{P_2, P_3\}$	56	16
P_3, P_1, P_2	15 $\mathcal{S} = \{P_3\}$	5	70
P_3, P_2, P_1	18 $\mathcal{S} = \{P_3, P_2\}$	2	70
$\phi_i(N, v)$	39.2	20.7	30.2

Example 2



- A set of (at least 3) firms producing gloves have formed a coalition.
- Each can produce either only left or only right gloves.
- At the time of selling the gloves, one must match exactly one right glove with exactly one left glove.
- Unmatched gloves are worth zero on the market.
- How can one fairly redistribute the revenue?

Glove game



$$v(1)=v(2)=v(3)=0$$

$$v(1,2)=0$$

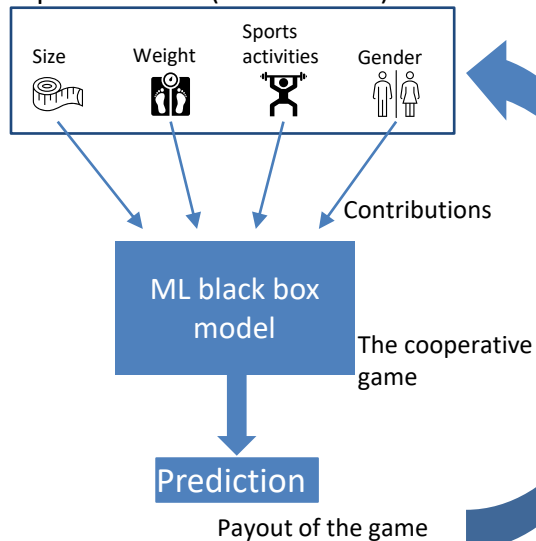
$$v(1,3)=v(2,3)=v(1,2,3)=1$$

Permutation	1	2	3
1,2,3	0	0	1
1,3,2	0	0	1
2,1,3	0	0	1
2,3,1	0	0	1
3,2,1	0	1	0
3,1,2	1	0	0
Sum	1	1	4
$\phi(v)$	1/6	1/6	4/6

SHapley Additive exPlanations (SHAP) Value

Shapley value for XAI

Input variables (contributors)



How to distribute the Payout fairly among the contributors?

- We treat each feature like a player in a game and calculate Shapley values to find out the contribution in the black box model.
- We want to compare how the coalition (black box model) would perform with versus without a specific player (input variable) – and this for all sub-coalitions → Shapley value.
- Shap is a local explanation technique. It aims to explain individual predictions of models.
- Shap decompose a prediction as follow:
$$pred\ with\ all\ features = sum(SHAP\ values\ for\ each\ feature) + pred\ for\ baseline\ values$$

→ the SHAP values of all features add up to explain why my prediction was different from the baseline.
- It is also possible to get valid global explanation through aggregating over these individual predictions.

Calculating SHAP

- Let
 - h an input/output ML predictive model.
 - $x_i = (x_{i1}, x_{i2}, \dots, x_{in})^T \in \mathbb{R}^n$ be the observation number i .
 - n be the total number of input variables in x_i .
 - $x_i \setminus \{x_{ij}\} = (x_{i1}, \dots, x_{i(j-1)}, x_{i(j+1)}, \dots, x_{in})^T$ be the observation number i without variable j .
 - $z \subseteq x_i \setminus \{x_{ij}\}$ be the observation number i with a subset of variables selected in $x_i \setminus \{x_{ij}\}$.
 - $|z|$ the number of variables in z .

$$\phi_j(h, x_i) = \frac{1}{n!} \sum_{z \subseteq x_i \setminus \{x_{ij}\}} |z|! (n - |z| - 1)! [h([z, x_{ij}]) - h(z)]$$

How do we exclude a feature from a machine learning model? Typically, the input are fixed, and we cannot just remove a variable of x because then the shape of input would. For the feature we want to exclude, we just input random values from the training set.

For instance: if $x_i = (x_{i1}, x_{i2}, x_{i3}, x_{i4})^T$ and $z = (x_{i1}, x_{i3})^T$ then concretely, to remove x_{i2} and x_{i4} , we will replace the values with random values on these columns selected in the training set.

Calculating SHAP

2^n = total number of
subset to calculate

The Shapley value can be approximated

- Kernel Shap
 - Tree Shap
 - Deep Shap
- } Model agnostic
- } Faster and more accurate but not model agnostic

Code Example

```
row_to_show = [6]
data_for_prediction = X_vl.iloc[row_to_show]

print('The data for prediction:')
print(data_for_prediction)

print('\n\nThe prediction:', reg.predict(data_for_prediction)[0])
```

✓ 0.7s

The data for prediction:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
7113	5.0839	36.0	6.221719	1.095023	670.0	3.031674	33.89	-118.02

The prediction: 2.3390358700186686

```
import shap
```

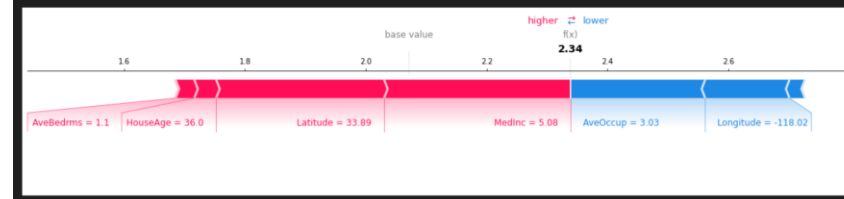
```
explainer = shap.TreeExplainer(reg)
shap_values = explainer.shap_values(data_for_prediction)
shap_values
```

✓ 0.3s

```
array([[ 0.30874621,  0.03620116, -0.02549494,  0.03090186,  0.00087705,
        -0.2226369 ,  0.27795437, -0.13945988]])
```

```
shap.initjs()
shap.force_plot(explainer.expected_value, shap_values, data_for_prediction.round(2), matplotlib=True)
```

✓ 0.7s



```
print(reg.predict(X_tr).mean())
print(explainer.expected_value[0]) # Base value
print(explainer.expected_value[0] + shap_values.sum())
```

✓ 0.2s

```
2.071946937378876
2.071946937378876
2.339035870018668
```

pred with all features = sum(SHAP values for each feature) + pred for baseline values

Code Example (KernelExplainer)

```
from sklearn import linear_model

lin_regr = linear_model.LinearRegression()
lin_regr.fit(X_tr, y_tr)

explainer = shap.KernelExplainer(lin_regr.predict, X_tr.sample(100))
shap_values = explainer.shap_values(data_for_prediction)
shap.force_plot(explainer.expected_value, shap_values, data_for_prediction.round(2), matplotlib=True)
```

✓ 0.9s

100% | 1/1 [00:00<00:00, 4.52it/s]

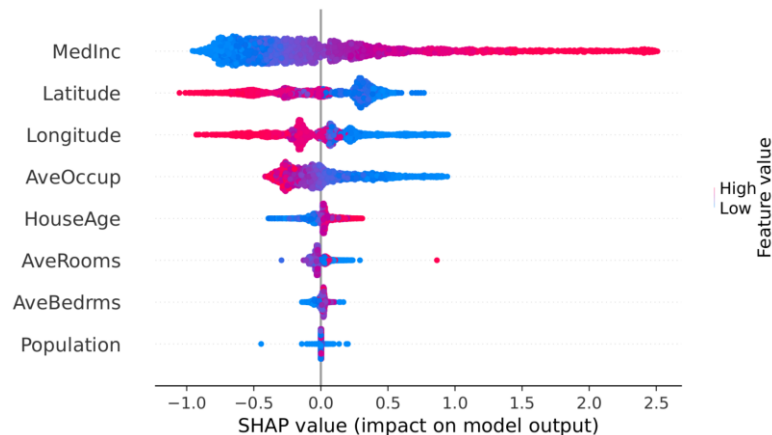


Global explainable ML with SHAP

- Permutation Importance is great because it created simple numerical measurements to see what features mattered for a model. It makes easy to do comparison between features and the resulting graphs can be presented to a non-technical audience.
- But that doesn't tell you how important each feature is for individual observations. If a feature has medium permutation importance, that could mean it has:
 - a large effect for a few predictions, but no effect overall, or
 - a medium effect for all predictions.
- The SHAP summary plots give us some additional insight into the importance of features and what drives them.

```
explainer = shap.TreeExplainer(reg)
shap_values = explainer.shap_values(X_vl)
shap.summary_plot(shap_values, X_vl);
```

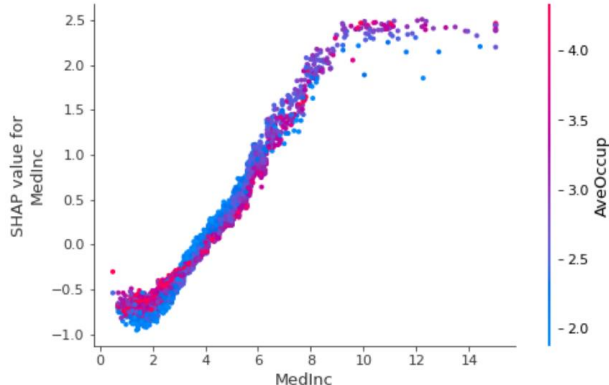
✓ 1.8s



SHAP Dependence Contribution Plots

- We've used partial dependency plots (PDP) before to show how a single feature affects predictions.
- These are insightful and relevant to many real-world use cases. Also, with a little effort, they can be explained to a non-technical audience.
- The SHAP dependence contribution plots provide a similar insight to PDP's, but they add a lot more detail.

```
explainer = shap.TreeExplainer(reg)
shap_values = explainer.shap_values(X_vl)
shap.dependence_plot('MedInc', shap_values, X_vl)
```



- Each dot is a single prediction (row) from the dataset.
- The x-axis is the value of the feature (MedInc).
- The y-axis is the SHAP value for that feature, which represents how much knowing that feature's value changes the output of the model for that sample's prediction. For this model the units are in \$100,000.
- The color corresponds to a second feature that may have an interaction effect with the feature we are plotting (by default this second feature is chosen automatically).